

Sistemas Distribuídos (DCC/UFRJ)

Módulo 4 - Parte 2 - Aula 2: Chamada remota de procedimento (RPC)

Profa. Silvana Rossetto

- Funcionamento básico RPC
- Passagem de parâmetros
- Suporte a aplicações baseadas em RPC
- Protocolo RPC e geração de stubs

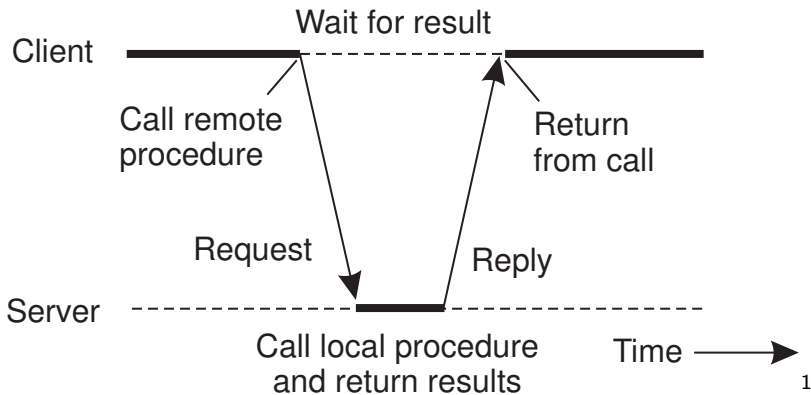
Chamada remota de procedimento (RPC)

- Permite que programas chamem funções remotas (localizadas em outras máquinas)
- O processo que chama a função fica bloqueado até que o resultado seja recebido
- Os dados trocados são passados como argumentos e retorno da chamada
- Toda a troca de mensagem (send/recv) fica oculta para o programador

Objetivo de tornar as chamadas remotas iguais a uma chamada de função local:

- do lado de quem chama a função remota, uma versão diferente dessa função é oferecida de forma “oculta”, chamada **stub do cliente**
- é responsável por **empacotar os argumentos da função** em uma mensagem e enviá-la para o servidor remoto
- em seguida, chama a **primitiva bloqueante de recebimento** de mensagem

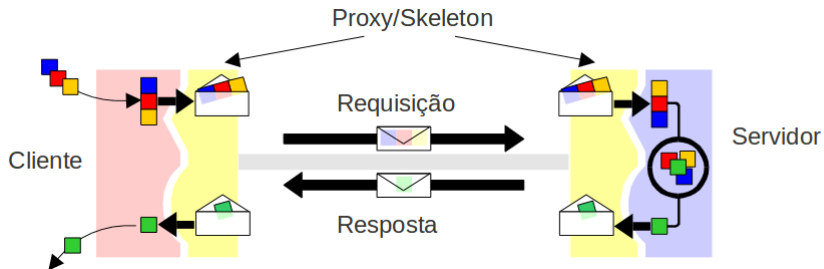
Funcionamento básico de RPC



¹Fonte: Steen-Tanenbaum, *distributed-systems.net*.

- do lado de quem recebe a chamada de função, a mensagem recebida é entregue para o **stub do servidor**
- é responsável por **desempacotar os argumentos** que estão na mensagem e chamar a função correspondente
- em seguida, **empacota os resultados** em uma nova mensagem e a envia de volta para o cliente

Funcionamento básico de RPC



O **empacotamento/desempacotamento** de parâmetros faz a transformação dos dados para uma **representação neutra**:

- do outro lado é **necessário entender o significado correto da sequência de bytes** recebida
- uma dificuldade é como passar **referências para ponteiros**
- uma alternativa é **copiar a estrutura de dados inteira e restaurar o valor na volta**
- outra alternativa é usar **referências globais** (ex., um arquivo em um mesmo sistema de arquivo)

O suporte a aplicações RPC pode se dar de duas formas:

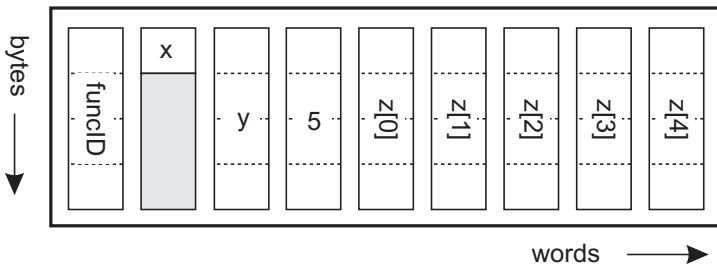
- ① por meio de uma **IDL** (linguagem de definição de interface) e de um **compilador** dessa linguagem que **gera os stubs cliente e servidor** para **diferentes linguagens**
- ② **embutindo a abstração de RPC** em **linguagens de uso geral** (ex., JavaRMI, RPyC)

O **empacotamento/desempacotamento** dos dados em/de mensagens requer a definição de um **protocolo RPC**:

- descreve de que forma os dados serão transmitidos (**tipos e formatos de mensagens**)
- define a **forma de representação das estruturas de dados** elementares (int, char, float, bool) (ex., **complemento a 2**, **unicode 16-bits**, **IEEE 754** e **little endian**)
- define o **protocolo de camada de transporte**

Exemplo de empacotamento de chamada RPC

```
void someFunction(char x; float y; int z[5])
```



Com protocolo RPC bem definido, os stubs cliente/servidor podem ser implementados:

- para um mesmo protocolo RPC, pode-se reutilizar os stubs para diferentes aplicações, variando-se apenas o conjunto de funções chamadas/oferecidas

A partir da **especificação das interfaces das aplicações** por meio de uma IDL, e do **protocolo RPC**, **stubs cliente e servidor são gerados automaticamente**

- o programador precisará **implementar apenas o núcleo das funções** do lado do servidor e a aplicação cliente que chama essas funções

M. van Steen and A.S. Tanenbaum, “Distributed Systems”, 3rd ed., www.distributed-systems.net, 2017

- **seção 4.2 (pag. 173-183 (saltar nota 4.5))**